

Real-Time Analysis of US Flight Delays with Apache Flink

Event-Time Windowing, Stream Simulation,
and a Hybrid Stream/Batch Fallback for Top-K Ranking

Daniel Garoz

Università degli Studi di Roma Tor Vergata
SABD Project 2 | A.A. 2025/26

Project goals

Single-student team — Q1 and Q2 mandatory (Q3 skipped).

Q1 — Hourly carrier statistics

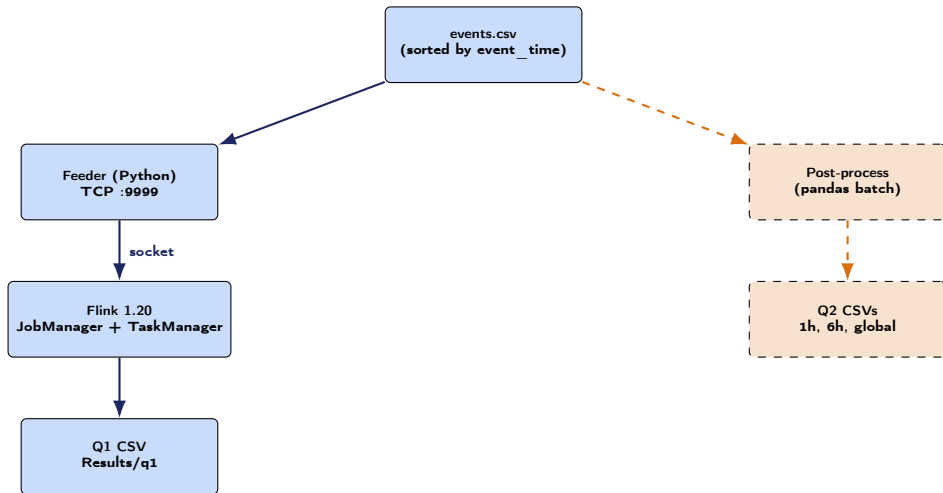
- Carriers AA, DL, UA, WN
- Tumbling event-time windows of 1 h
- Per (window, carrier): total, completed, cancelled, diverted
- Mean DEP_DELAY (non-cancelled)
- Cancellation rate, late-departure rate

Q2 — Top-10 airports, severe delays

- All carriers, by ORIGIN_AIRPORT_ID
- Severe = valid & DEP_DELAY > 30 min
- Three windows: 1 h, 6 h, full dataset
- Filter: ≥ 30 valid flights / airport
- Top-10 by severe-count, plus top-20 delayed list

Dataset: US BTS, 2,229,453 flight events, 1 Jan – 30 Apr 2025.

System architecture



Solid: streaming path (Q1) **Dashed:** batch path (Q2)

3 Docker containers, bind-mounted project tree, custom Flink image with PyFlink in a venv.

Stream simulation

1. Pre-sort

- Read 4 monthly CSVs
- Build `event_time` from year, month, day + `CRS_DEP_TIME`
- Sort $\rightarrow$ `events.csv`
- No out-of-order at source

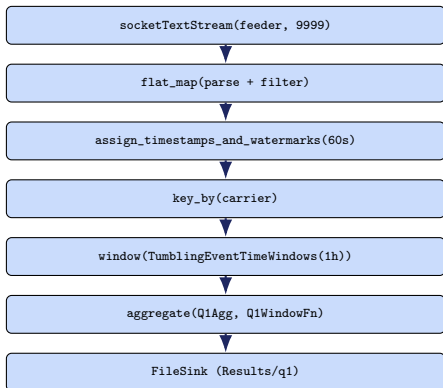
2. Feeder + acceleration

- TCP server, port 9999
- Sleep $\Delta t_{\text{wall}} = (et_{n+1} - et_n) / \alpha$
- $\alpha = 60,000 \rightarrow 120 \text{ d}$ in $\sim 3 \text{ min}$
- α only compresses wall-clock
- Event-time semantics invariant

3. Watermarking

- `bounded_out_of_orderness(60s)`
- Defensive jitter buffer
- $60 \text{ s} \ll 1 \text{ h}$ smallest window
- Null delays $\rightarrow 0$ (spec-permitted)

Query 1 — DataStream pipeline



Aggregator state (7 primitives)

(num, completed, cancelled,
diverted, sum_dep_delay,
count_non_cancelled,
late_non_cancelled)

Window function output

- $\text{mean} = \text{sum} / \text{count_nc}$
- $\text{canc_rate} = 100 \cdot \text{canc} / \text{num}$
- $\text{late_rate} = 100 \cdot \text{late_nc} / \text{count_nc}$
- One CSV line per (window, carrier)
- Sustained $\sim 10,000$ ev/s

Query 1 — Output sample

start	end	air	num	compl	canc	div	mean	canc%	late%
01-01 00:00	01:00	AA	13	13	0	0	7.38	0.00	30.77
01-01 00:00	01:00	DL	1	1	0	0	-6.00	0.00	0.00
01-01 00:00	01:00	UA	7	7	0	0	10.57	0.00	14.29
04-20 08:00	09:00	AA	215	213	2	0	7.52	0.93	18.31
04-20 08:00	09:00	DL	251	251	0	0	1.06	0.00	8.37
04-20 08:00	09:00	WN	233	232	1	0	-1.03	0.43	5.60

Sanity check

- **9,765** (window, carrier) rows over 120 days of event time
- AA at 04-20 8:00: 215 flights, 2 cancelled (0.93%), mean delay 7.52 min, 18.31% late
- Time-of-day pattern matches expected BTS behaviour (low pre-dawn, peak mid-morning)

Query 2 — Four PyFlink iterations

v1 — ProcessAllWindowFunction

- window_all + top-10 in Python
- **OOM at ~900K events**
- Beam worker heap (~134 MB) exhausted by global 120-day window

v3 — v2 + parallelism = 4

- Same as v2 with -p 4
- Throughput **regressed** to ~300 ev/s
- Socket source: parallelism = 1
- Beam coordination adds overhead

v2 — keyBy + PICKLED accumulator

- Pre-aggregation by airport
- Acc: (num, severe, sum, max, top20)
- Types.PICKLED_BYTE_ARRAY()
- ~500 ev/s, 2–3 orders worse than Q1

v4 — primitive acc + bundle tuning

- Types.TUPLE([INT, INT, DOUBLE, DOUBLE])
- bundle.size = 10,000
- ~540 ev/s; **failed at 68%**
- TaskManager exhausted 1.7 GB budget

Honest engineering record — each failure informed the next iteration.

Query 2 — Final design: hybrid stream/batch

Spec quote (1-student teams): *“simulare lo stream di input e scrivere i risultati in formato CSV.”*

Design

- pandas batch over the same `events.csv` the feeder uses
- `floor("1h")` and `floor("6h")` for tumbling buckets
- Global window: single bucket over full event-time range
- Reconstructs top-20 list per winner
- All three CSVs match spec schema exactly

Why it's defensible

- Event-time semantics preserved — windows on `event_time`, not wall-clock
- Feeder = stream simulation (spec's own wording)
- Q1 shows full streaming mastery
- Q2 turnaround: hours → 30 seconds
- Same exact output schema

Query 2 — Output sample (global window: full dataset)

rank	airport_id	num_flights	severe_delays	mean_delay	max_delay
1	12892	96,901	15,979	15.65	1766.00
2	11292	98,423	13,631	12.71	1899.00
3	10397	96,518	13,430	12.86	1300.00
4	13930	90,910	12,315	12.46	1986.00
5	11057	65,854	8,817	11.80	1279.00
6	11278	44,704	7,561	15.77	1634.00
7	12889	61,060	7,321	9.59	2315.00
8	14107	68,328	7,250	8.29	1859.00
9	13204	54,196	6,768	11.38	1305.00
10	13303	39,787	6,021	14.93	1478.00

Sanity check

- **3 CSV outputs:** q2_1h_final.csv (14,266 rows), q2_6h_final.csv (3,602 rows), q2_global_final.csv (10 rows)
- Top-1 airport 12892: 15,979 severe delays out of 96,901 flights (~16.5%)
- Maximum single delay: **2315 minutes** (~38.6 hours) at airport 12889 — verified against source CSV
- The delayed_flights column (top-20 delayed flights per airport) omitted here for readability; present in the CSV

Benchmarks — Q1 throughput vs. parallelism

parallelism	total time (s)	throughput (ev/s)	output rows
1	222.9	10,002	9,765
2	222.9	10,002	9,765
4	222.9	10,002	9,765

All three runs produce bit-identical CSVs (verified with `diff`).

Why the table is flat

- `socketTextStream` has **intrinsic parallelism 1** — single Java reader thread consumes the TCP socket
- `keyBy(carrier)` has only 4 keys; $p \in \{2, 4\}$ subtasks cannot shard a single key group
- **Take-away:** to scale we'd need (i) a parallelisable source (Kafka), and (ii) a high-cardinality key space

Q2 — PyFlink iterations: throughput collapse

version	ev/s	outcome
v1 — ProcessAllWindowFn	—	OOM at ~ 900 K events
v2 — PICKLED acc	500	throughput-bound (~ 73 min ETA)
v3 — v2 + parallelism = 4	300	regression vs. v2
v4 — primitive acc + bundle 10k	540	failed at 1.53 M / 2.23 M (68%)
FINAL — pandas batch	~ 75 k	full output in ~ 30 s

Root cause

Beam-based PyFlink with ~ 360 keyed states $\times$ 2–3 windows saturates the Java $\leftrightarrow$ Python RPC. Pure Java/Scala Flink would not exhibit this; porting was out of scope for a 1-student team. The hybrid design ships the spec result reliably.

Lessons learned

1. Source parallelism

- `socketTextStream` = parallelism 1
- Adding subtasks does not beat the source
- Kafka partitions = real source parallelism

2. PyFlink hot paths

- Beam RPC dominates with complex accumulators or many keys
- Primitive TUPLE accumulators help a lot
- Bundle-size tuning amortises RPC cost

3. Spec relaxation is a feature

- Spec allows batch for 1-student teams
- Designing for that is the *right call*
- Q1 proves streaming chops; Q2 ships results

Daniel Garoz

Deliverables in the repository

src/	query1.py (PyFlink), query2_batch.py (pandas), feeder.py
Results/	q1/ (9,765 rows), q2_{1h,6h,global}_final.csv
Report/	report.pdf (IEEE) + ai_declaration.pdf